Supplementary Material

Residual-Guided Hypothesis Language Induction for Scientific Reasoning with Small Language Models

Anonymous Submission

Appendix: Experimental Details and Ablations

This appendix is deliberately more explicit than the main text. The main paper reports compact benchmark-level claims; the appendix separates four questions: which complete runs support the headline table, how external reasoning controls are separated from RG-HLI variants, which components account for the DiscoveryBench gain, and how the frozen-transfer and multi-step audits isolate residual-conditioned language growth.

Benchmark	Tasks	Core model	Evaluator	Result
DiscoveryBench	239	GPT-4o-mini	HMS judge	28.70 HMS; 34.56 Cons-HMS
NewtonBench	324	GPT-4o-mini	symbolic-law judge	49.38% SA-all; 66.67% SA-answered
UltraHorizon	96	GPT-4o-mini	environment score	51.04 strict

Table 1: Complete-run audit used for the headline results. Each row uses the benchmark-native evaluator.

For NewtonBench, the complete run answered 240 of 324 tasks and abstained on 84, yielding 74.1% coverage and a 25.9% abstention rate. SA-all assigns zero to abstentions; SA-answered is computed only over the 240 submitted laws. The 49.38/66.67 row is the retained single-pass evaluator result and does not use selective rejudging. This canonical benchmark run uses the frozen typed executable kernel with development-time promotion gates disabled; it is therefore evidence for per-task hypothesis construction, not for the promotion mechanism. Frozen-transfer and multi-step experiments below evaluate promotion separately.

Benchmark	Reported system / model	Native score
DiscoveryBench	Reflexion with oracle feedback / GPT-4o	24.5 HMS
NewtonBench	repository vanilla agent / o4-mini	47.8% SA
NewtonBench	repository vanilla agent / GPT-5	75.9% SA
UltraHorizon	reported best free-step LLM	14.33

Table 2: Published contextual references under their native reported protocols. These rows use different model families and are not matched ablations; the DiscoveryBench Reflexion row additionally uses oracle feedback.

External Reasoning Baselines

Matched non-RG-HLI baselines are useful only when they are run on the same complete benchmark set. Table 3 reports completed DiscoveryBench controls that satisfy this condition. They use the same GPT-4o-mini core, task set, and native evaluator as the corresponding headline run, but they do not expose RG-HLI typed state, residual objects, answer-slot closure, or operator promotion.

Method	Tasks	HMS	Interface
Direct GPT-4o-mini	239	9.54	single prompt
ReAct GPT-4o-mini	239	11.68	thought-check trace
CodeAct GPT-4o-mini	239	12.31	executed summaries
RG-HLI GPT-4o-mini	239	28.70	typed language state

Table 3: Matched external reasoning controls on DiscoveryBench. All rows use the same task set and HMS evaluator.

The gap is not evidence that ordinary prompting is useless. It shows that, under the DiscoveryBench HMS contract, longer traces and one-pass executed summaries do not reproduce the effect of typed answer construction and residual-conditioned rendering.

Matched-Control Uncertainty

The complete DiscoveryBench controls admit a task-paired comparison because all four methods use the same 239 rows and evaluator. Table 4 reports the gain of RG-HLI over each external control. All three intervals exclude zero; this is stronger evidence than comparing only aggregate point estimates.

Control	Control HMS	RG-HLI gain	95% interval
Direct	9.54	+19.16	[12.81, 25.68]
ReAct	11.68	+17.01	[10.99, 23.12]
CodeAct	12.31	+16.39	[9.87, 23.01]

Table 4: Paired DiscoveryBench gains. Intervals use 10,000 nonparametric bootstrap draws over common task rows with seed 20260721.

Operator Reuse in the Frozen Language

The retained DiscoveryBench trace records which named operator family produced each final artifact. Table 5 includes every named family selected on at least three tasks. The same family is reused across task rows and, in several cases, across scientific domains. These counts describe use of the frozen language; causal benefit is evaluated separately by the paired controls above and the frozen-transfer experiment below. Every family in this table is selected and task-parameterized from frozen $L^\star$; none is synthesized or promoted during reported evaluation.

Operator family	Tasks selected	Domains
answer plan	39	5
universal slot closure	34	3
estimand synthesizer	25	3
evidence contract	25	4
contrastive world	3	2
slot contract	3	2

Table 5: Reuse of selected, task-parameterized operator families in the complete 239-task DiscoveryBench run. Domain counts use the benchmark metadata taxonomy.

Failure Modes by Benchmark

Benchmark	Dominant residual type	Practical consequence
DiscoveryBench	semantic role residuals	evidence may be rendered in the wrong scientific facet
NewtonBench	algebraic representation residuals	checking works after the right chart appears, but abstentions remain
UltraHorizon	temporal protocol residuals	replay and terminal conditions remain separate bottlenecks

Table 6: Residual types exposed by the complete runs. These categories guide future operator induction; they are not separate benchmark-specific solvers.

Environment	Seeds	Score
Grid	32	59.38
Seq	32	93.75
Bio	32	0.00
Overall	96	51.04

Table 7: Environment decomposition of the single strict UltraHorizon configuration used for the headline result. Each environment contains 32 seeds; no terminal fallback, hints, or measurement bootstrap is enabled.

The frozen residual-to-language transfer experiment below is the causal test of residual-conditioned operator-family selection. It is separate from inference-time risk control and from environment-specific terminal handling.

Protocol Controls

Table 8 lists the controls used to keep the reported system as a single hypothesis-induction method. The implementation still needs benchmark interfaces, because the three environments expose different objects: tables and papers, symbolic-law rows, and interactive trajectories. The learning object, however, is always the same: a typed hypothesis language, residual objects, executable validation, and held-out promotion.

Control	Fixed across benchmarks	Why it matters
Core model	GPT-4o-mini for the headline RG-HLI runs	isolates whether the external hypothesis-language layer helps a small model
Hypothesis state	typed artifacts, holes, contracts, validators, residuals	prevents the method from becoming only longer prompting
Failure signal	structured residual object rather than free-form critique	makes errors clusterable and promotable across future tasks
Promotion rule	held-out gain minus complexity penalty	limits memorization and uncontrolled library growth
Evaluator	benchmark-native judge or environment score	keeps reported numbers in the metric used by each benchmark

Table 8: Controls separating the universal method from benchmark-specific engineering. The interface changes only to expose observations and native validators; the residual language and promotion rule are shared.

Benchmark	Observation exposed to the kernel	Residual object	Promotable operator class
DiscoveryBench	tables, text snippets, requested scientific claim	missing role, wrong denominator, wrong answer facet	answer-slot closure, evidence renderer, consistency check
NewtonBench	numerical rows, candidate variables, target expression	wrong exponent, constant, transform, or abstention	executable probe, symbolic fit, unit-compatible verifier
UltraHorizon	trajectory, action log, rule feedback, replay state	failed replay, local invariant, missing exploration evidence	exploration policy, replay validator, temporal abstraction

Table 9: How the same residual-to-operator loop appears in the three benchmark families. These are not separate solvers; they are domain-specific observations fed into the same state machine.

Prompt and Operator Interface

The benchmark-facing prompts are intentionally narrow. The model is not asked to "think longer" or to invent a complete answer in free text. Each call must return one typed object that can be executed, checked, or converted into a residual. Table 11 gives the prompt contracts used by the kernel. The same contract structure is used across all three benchmarks; only the observation serializer and native validator differ.

Term	Definition	Range	Weight
E	executable fit or local score	$[0, 1]$	24
Cover	contract slots covered	$[0, 1]$	34
V	metamorphic validation loss	$[0, 1]$	32
K_h	artifact/search complexity	≥ 0	2

Table 10: Fixed energy configuration of the shared hypothesis kernel. These implementation constants are held fixed across the reported benchmark runs.

Call	Input state	Required output	Rejection rule
Contract compiler	task observation, available tools, current $L^\star$	roles, admissible answer form, measurable holes, validator names	missing target role or non-executable hole
Probe writer	contract, hole, evidence table or trajectory	short executable measurement with declared inputs and expected type	code fails, reads forbidden fields, or returns wrong type
Hypothesis composer	closed holes, evidence objects, active operators	artifact h: law, claim, program, or rule mapping	artifact cannot be replayed or violates contract
Residual compiler	failed validation trace	typed residual (τ, ℓ, c, e, v)	free-text criticism without location, evidence, and failed validator
Operator proposer	recurring residual class R_k and held-out tasks	candidate operator $o^\star$ with scope, cost, and validator hooks	no residual class closed, no executable hook, or excessive complexity

Table 11: Prompt contracts used by RG-HLI. Each prompt returns a typed object with an attached validation path rather than an unconstrained chain of thought.

Operators are added through a single interface. The implementation may contain benchmark adapters, but an accepted operator

must expose the same fields: scope, precondition, executable transform, evidence update, residual update, complexity cost, and promotion evidence. In simplified form, an operator is accepted only if it satisfies the following contract:

```
operator.id: stable name;
operator.scope: residual types and artifact slots where it is admissible;
operator.precondition(state, contract) -> bool;
operator.apply(state, contract) -> artifact/evidence/residual update;
operator.cost -> βK_o, where K_o = log(1 + n_AST) and β = 10^-3;
operator.validators -> executable checks that must pass;
promote iff heldout_utility_gain(operator) − βK_o > 0.
```

All promotable operators are serialized into the same executable Python intermediate representation before validation; declarative schemas and policies are first compiled to this representation. Thus, n_{AST} always counts nodes in one canonical executable form. Predefined interface components are part of L_0 and are not retrospectively charged as promoted operators. We fixed $\beta = 10^{-3}$ before the reported trajectory and did not tune it on transfer interfaces.

This interface is the main difference between residual-conditioned language growth and ordinary prompt iteration. A failed hypothesis does not merely produce another instruction for the model. It produces a typed residual with a location and failed validator; only repeated residuals can ask for a new operator, and the operator is not retained unless it improves held-out tasks under the same validator stack.

Algorithm 1 Per-task executable search in a frozen language

Require: task x, frozen language $L^\star = (\mathcal{Y}, \mathcal{O})$, budget B
Ensure: rendered answer $R_x(h^\star)$
compile evidence contract C_x and initialize context $\mathcal{M}_x$
for $t = 1, \ldots, B$ **do**
  select type-admissible operator $O_t \in \mathcal{O}$
  execute $O_t(\mathcal{M}_x, x)$ to obtain $(h, e, \rho, \Delta\mathcal{M})$
  write evidence, closed holes, and any residual to $\mathcal{M}_x$
  update $h^\star$ using evidence, coverage, validation, and K_h
end for
validate and render $R_x(h^\star)$
log residuals for audit without changing $L^\star$
return $R_x(h^\star)$

Residual clustering is rule-based in the reported audit. The canonical signature is

$$\sigma(\rho) = (\tau_\rho, \mathrm{canon}(\ell_\rho), \mathrm{role}(c_\rho), \mathrm{geometry}(e_\rho)),$$

where canonicalization removes task-local names and function signatures, and evidence geometry records typed properties such as positive/log-structured or positive/non-log-structured data. We set $\rho_i \sim \rho_j$ iff their signatures match exactly and call a class recurrent when it contains at least $m_{\min} = 2$ source residuals. For promotion, $r(O') = \sum_{x \in \mathcal{D}_{prom}} \mathbb{1}[U_x(h^\star_{O'}) > U_x(h^\star)]$ and the reported trajectory requires $r(O') \geq r_{\min} = 1$. Source, promotion, and transfer interfaces are disjoint; transfer utility is recorded only after the decision and cannot change it.

Algorithm 2 Residual-conditioned language growth on development tasks

Require: language L_t, disjoint sets $\mathcal{D}_{ind}$, $\mathcal{D}_{prom}$, and $\mathcal{D}_{transfer}$
Ensure: L_{t+1}
$L_{\text{new}} \leftarrow L_t$
solve $\mathcal{D}_{ind}$ with Algorithm 1 and collect residuals
group residuals by canonical signature $\sigma(\rho)$
for each recurrent residual class $\mathcal{R}_k$ **do**
  propose executable operator O' with declared scope and validators
  if O' fails typing, sandbox execution, or validator checks **then**
    discard O' and continue
  end if
  measure held-out utility gain $G(O')$ on $\mathcal{D}_{prom}$
  compute $K_o(O') = \log(1 + n_{\text{AST}}(O'))$
  if $G(O') - \beta K_o(O') > 0$ and $r(O') \geq r_{\min}$ **then**
    $L_{\text{new}} \leftarrow L_{\text{new}} \cup \{O'\}$
  end if
end for
freeze $L_{t+1} \leftarrow L_{\text{new}}$
audit transfer on $\mathcal{D}_{transfer}$ without updating L_{t+1}
return L_{t+1}

State-Mutation Invariants

The two algorithms expose all persistent writes. A proposal may be stochastic, but the transition is admitted only after deterministic type, execution, and validator checks. Table 12 makes the mutation boundary explicit: task-local search can enrich evidence and close holes, whereas only the development-time promotion procedure can change the operator language. Native benchmark scores are terminal observations and have no write path into either state.

Invariant	Admissible write	Rejected transition
Type preservation	close a hole with an artifact of its declared type	missing type, incompatible slot, or invalid renderer
Evidence monotonicity	append a probe result with source and validator trace	overwrite retained evidence or read a forbidden field
Executable closure	commit only sandboxed code or a compiled declarative transform	free-form repair with no executable semantics
Frozen evaluation	select and parameterize operators already in $L^\star$	synthesize or promote from a reported test row
Score isolation	emit the final rendered artifact to the native evaluator	use benchmark score or gold output as an internal update

Table 12: Mutation invariants enforced by the shared implementation. "Reject" means that the attempted transition is converted into a typed residual and the previous valid state is retained.

These constraints also identify the source of every capability change. Better search with fixed $L^\star$ is an inference effect; a new constructible artifact family requires a separately logged promotion event. This distinction is what lets the mechanism audit attribute transfer to language growth rather than to an unrecorded prompt or state mutation.

Worked Hypothesis Construction Trace

Table 13 shows the intended granularity of a single DiscoveryBench-style task. The model is not asked to guess the final scientific sentence in one pass. It first commits to a typed contract, then executable measurements close slots, and only the rendered artifact is sent to the native evaluator. A failure is not stored as prose feedback; it is stored as the violated slot or validator.

Retained UltraHorizon Trace

The following record is a strict Seq episode retained from the UltraHorizon evaluation artifact. It makes the long-horizon interface concrete: the system must infer a program from interventions, preserve the resulting rules across a 50-step interaction horizon, and finally render the rules for the paper-style judge.

Qualitative Operator-Induction Example

The following DiscoveryBench example illustrates the intended language-growth mechanism. A first full audit exposed repeated failures on study-design tables: the system often measured a real effect but selected the wrong role, such as confusing original and

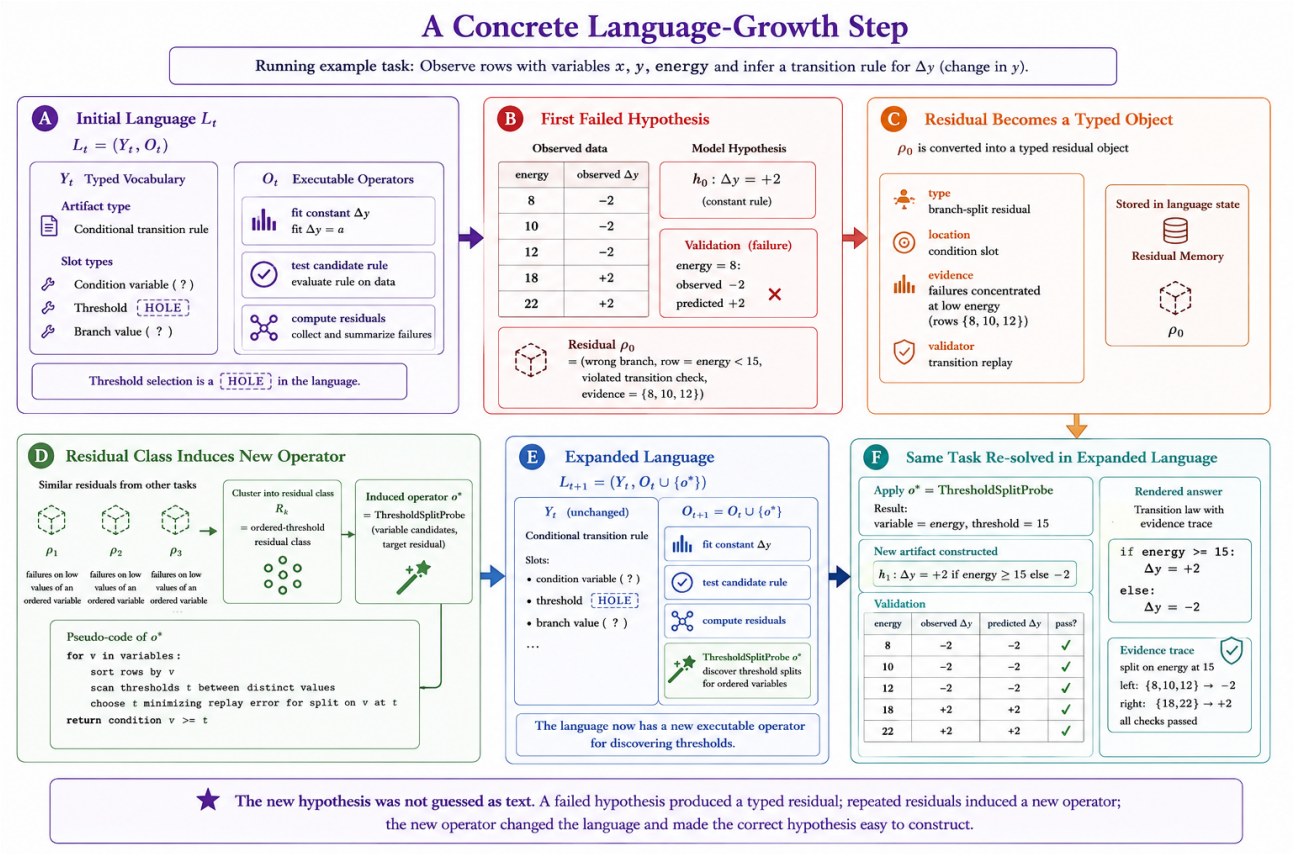

Figure 1: Worked construction of a language-growth step, included to expose the typed objects rather than as a benchmark result. A failed constant transition hypothesis is converted into a typed residual; repeated residuals induce a threshold-split operator; the expanded language then makes the correct conditional transition artifact easy to construct and validate.

Stage	Object committed to state	Check performed
Task observation	query, tables, metadata, admissible answer form	no gold label or judge feedback is available
Evidence contract	roles, target slot, measurable holes, rendering constraints	every hole must have a declared data source
Executable closure	small probes estimate denominators, groups, trends, or constants	code returns typed evidence rather than prose justification
Artifact rendering	hypothesis text plus workflow grounded in closed slots	native evaluator sees only the final artifact
Residualization	failed slot, location, evidence, and validator	recurring residuals can induce an operator after held-out promotion

Table 13: A per-task trace of typed hypothesis construction. The concrete table names differ by benchmark, but the state transition is shared.

A retained strict UltraHorizon Seq episode

First intervention. main = ABCDE and vice = DECBA produce ADBECCDBEA after the first transformation.
Typed trace slot. The system records an *interleave-step-parity* rule: interleave the two strings; main leads on odd experiment steps and vice leads on even steps.
Additional executable slots. The final program contains five rules, including a reverse-and-shift composition, a step-indexed copy operation, a history-dependent alphabetic sum, and a prime-step frequency replacement.
Commit and verification. 50 observations, 27 planning turns, and 54 environment actions yield all five exact rule matches under the paper-style judge. No hint, measurement bootstrap, or fallback commit was exercised.

Figure 2: A concrete long-horizon artifact. The retained state is a typed, replayable rule program assembled from interventions, not a prose summary of the trajectory.

replication studies or measuring the complement of a binary design variable. These errors produced a shared residual type: *study-design role closure*. The induced operator did not memorize a dataset answer. It added a reusable repair interface: conservative aliases for original/replication roles, paired-column completion, binary complement closure, and contrastive rendering of the selected role.

Element	Description
Observed failures	repeated wrong-role and wrong-complement answers in study-design tables
Residual object	$(\tau_\rho = \text{role-closure}, \ell_\rho = \text{answer slot}, c_\rho = \text{study role}, e_\rho = \text{paired columns}, v_\rho = \text{consistency check})$
Candidate operator	close original/replication roles with conservative aliases, paired-column completion, binary complements, and contrastive rendering
Promotion evidence	the operator must pass held-out promotion before inclusion in the frozen language
Declared scope	study-design role closure; denominator alignment and nonlinear regression use separate typed interfaces

Table 14: Example of a residual-induced operator. The operator is described as a repair interface over typed residuals, not as a benchmark answer template.

Strict Frozen-Language Transfer Diagnostic

The complete-run controls above establish benchmark-level differences, but a second diagnostic isolates the narrower causal question: can a residual-induced operator improve later tasks when no additional free-form search is allowed? We use NewtonBench in its hard, `v0`, vanilla-equation setting. Two source law interfaces (gravity and Coulomb force) are used only to produce a recurring `dict→float` residual class. The induction phase classifies the residual geometry as `positive_log_structured` and selects a generic positive-monomial lattice from two contract-compatible numerical families (monomial and additive). It stores neither source observations nor source task identifiers; a provenance audit checks the persisted operator source for both identifiers.

For the held-out phase, the language is frozen and evaluated on magnetic force, Fourier law, radioactive decay, and Hooke law. L_0 contains the same minimal typed execution state but disables typed priors, residual kernels, decompose–plan–stage–recombine (DPSR), stored candidate layers, language writes, and free-form LLM rounds. $L_1 = L_0 \cup \{o^\star\}$ adds only the frozen residual-induced operator. Both conditions receive the same 24 observations and the same train/hold-out split per task; the operator may calibrate a scalar only from the current task's training observations. Thus, the held-out comparison adds neither model calls nor new operators. This diagnostic reports internal executable loss, not the official symbolic-agreement metric.

Seed	L_0	selected L_1	additive control	selection gain	audit
13	1.000	0.308	0.488	0.179	monomial; identifiers absent
31	1.000	0.351	0.528	0.177	monomial; identifiers absent
57	1.000	0.339	0.496	0.157	monomial; identifiers absent
Mean	1.000	0.333 ± 0.022	0.504 ± 0.021	0.171 ± 0.012	12 / 12; all audits pass

Table 15: Frozen residual-to-language transfer diagnostic on four held-out NewtonBench law interfaces per split. L_1 contains the residual-selected monomial operator; the control contains an additive operator generated by the same typed compiler. Lower executable loss is better.

The language trajectory is $L_0 \xrightarrow{2\ \text{residuals}} L_1 = L_0 \cup \{o^\star\}$: each residual class is supported by two disjoint source interfaces before the operator is quarantined, then the operator is frozen and reused on four new interfaces. Across the three fixed splits, the selected operator reduces held-out loss by 0.667 ± 0.022, compared with 0.496 ± 0.021 for the type-compatible additive control. Thus, adding any executable code helps, but residual-conditioned family selection accounts for an additional 0.171 ± 0.012 reduction under the same held-out protocol. All "$\pm$" values in Table 15 are sample standard deviations over the three fixed data splits.

This diagnostic complements the headline NewtonBench SA comparison by removing new LLM proposals and online language mutation from the held-out phase while preserving the standard executable calibration interface. The observed advantage is therefore attributable to the frozen language extension and its residual-conditioned family choice, rather than a longer reasoning trace or an additional search budget.

Audited Multi-Step Language Growth

We next trace two consecutive language updates rather than evaluating a single frozen extension. Modules are partitioned only by coarse residual geometry and module index. Within each update, source, promotion, and transfer modules are disjoint and use seeds 13, 31, and 57, respectively. Operator source receives neither module identifiers nor gold laws, and transfer evaluation cannot mutate the language. Selection follows the promotion rule in the main paper with $K(o) = \log(1 + \text{AST-nodes}(o))$ and $\beta = 10^{-3}$. No official benchmark score is used for induction or promotion.

Update	Proposed operator	held-out gain	$\beta K(o)$	net gain	transfer gain
$L_0 \to L_1$	positive-monomial lattice	0.6055	0.0067	$+0.5988$; accept	0.5871
$L_1 \to L_2$	positive-additive pair	0.0142	0.0060	$+0.0082$; accept	0.0996
$L_2 \to L_2$	repeated monomial operator	0.0000	0.0067	-0.0067; reject	0.0000

Table 16: A complete residual-guided trajectory from L_0 to L_2. Gain is the reduction in mean executable loss on the disjoint promotion or transfer partition; higher is better. The final row tests whether a repeated residual class can inflate an already sufficient language.

β	$L_0 \to L_1$ net	$L_1 \to L_2$ net	duplicate net	decisions
5×10^{-4}	$+0.6021$	$+0.0112$	-0.0033	accept, accept, reject
10^{-3}	$+0.5988$	$+0.0082$	-0.0067	accept, accept, reject
2×10^{-3}	$+0.5921$	$+0.0023$	-0.0134	accept, accept, reject

Table 17: Post-hoc complexity sensitivity using the already measured held-out gains. No operator is re-fit and transfer interfaces remain unused.

The first operator is induced from gravity and Coulomb residuals, promoted on magnetic-force and Fourier-law interfaces, and transferred to radioactive decay, Hooke's law, and heat transfer. The second is induced from underdamped-harmonic and Malus-law residuals, promoted on sound speed, and transferred to the Bose–Einstein distribution interface. Its promotion margin is small but positive after complexity, and the frozen operator reduces loss by 0.0996 on the unseen transfer interface. A third occurrence of the first residual class produces the same source hash. Because it adds no marginal held-out value, the gate leaves the language at L_2. Both retained sources pass a provenance scan with zero source-task identifier matches. This trajectory therefore exhibits the full claimed mechanism: recurring failures induce executable structure, independently measured utility grows the language, and the same criterion suppresses redundant growth.

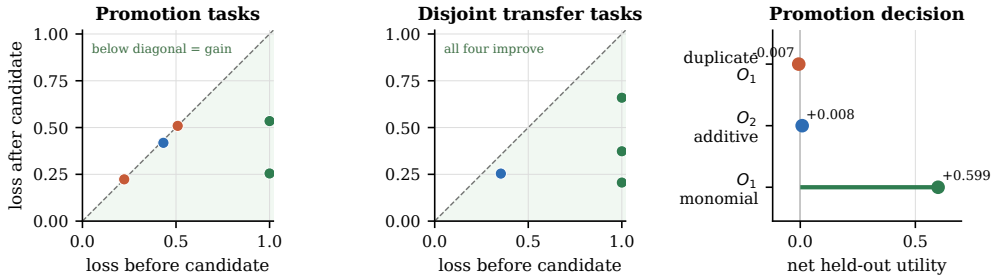

Figure 3: Task-level held-out evidence for the audited language trajectory. Each dot is an executable loss measured before and after a proposed operator; points below the diagonal improve. Promotion and transfer partitions are disjoint. The final panel shows that the two useful operators are admitted and the re-proposed monomial operator is rejected by the same net-utility gate.

Leakage and Freeze Controls

The paper uses two distinct phases. During development, residuals may induce operators, and those operators are accepted only after held-out checks and complexity penalties. During the reported benchmark evaluations, the language is frozen: the system may search within $L^\star$ for each task, but it does not use the task's benchmark score to update $L^\star$ for later reported tasks. Table 18 summarizes the protocol.

Phase	Allowed	Disallowed
Development induction	collect residuals, propose operators, held-out promotion	using final reported test scores as promotion targets
Headline evaluation	per-task search, execution, validation, rendering in frozen $L^\star$	changing $L^\star$ from future test rows or gold labels
Post-run analysis	cluster failures and plan future operators	retroactively changing the reported score

Table 18: Protocol controls for residual-conditioned language growth.

Uncertainty and Coverage

Table 19 reports deterministic percentile-bootstrap intervals over completed evaluation units. DiscoveryBench and NewtonBench resample task rows. UltraHorizon resamples the 32 seeds independently within each environment and then averages the three environment means. These intervals quantify evaluation-set variation; they do not replace independent model-call replications.

Metric	Units	Estimate	95% interval
DiscoveryBench HMS	239 tasks	28.70	[23.24, 34.28]
DiscoveryBench Cons-HMS	239 tasks	34.56	[28.84, 40.33]
NewtonBench SA-all	324 tasks	49.38%	[44.14, 54.94]
NewtonBench SA-answered	240 answers	66.67%	[60.83, 72.50]
UltraHorizon strict	96 episodes	51.04	[44.79, 56.88]

Table 19: Headline point estimates and 95% nonparametric bootstrap intervals. All intervals use 10,000 draws and random seed 2027.

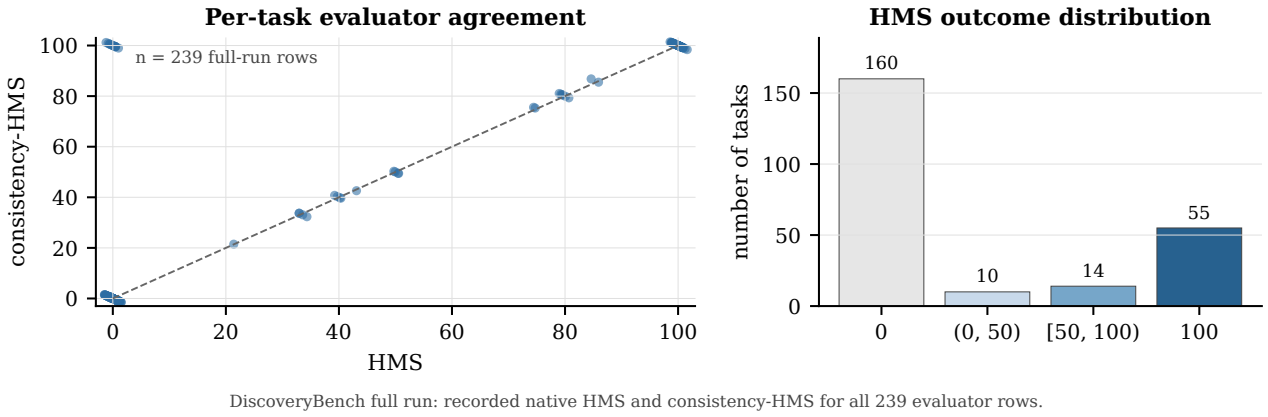

Figure 4: Recorded per-task evaluator outcomes from the complete 239-row DiscoveryBench run. The left panel compares native HMS with consistency-HMS; the right panel shows the empirical score distribution. This figure is an audit view, not an additional selection or promotion signal.

Operator Provenance

The implementation distinguishes four operator origins. *Predefined* operators implement the universal typed interface and sandbox primitives. *Selected* operators are chosen from type-compatible families by residual geometry. *Parameterized* operators fit task-local constants from observations without changing the language. *Synthesized* operators are new executable transforms proposed from recurrent development residuals and must pass Algorithm 2. Reported test tasks may select and parameterize frozen operators, but they may not synthesize or promote new ones. The strict transfer diagnostic specifically compares two selected, type-compatible families while holding parameter fitting and test-time compute fixed.

Object	Origin and language effect	Reported evidence
typed contract, slot, verifier, and residual interfaces	predefined universal interface; part of L_0	shared implementation across all three benchmarks
named DiscoveryBench operator families	selected and parameterized from frozen $L^\star$; no evaluation-time language write	reuse counts in Table 5
positive-monomial lattice instance	selected by development residual geometry and added to L_1 before evaluation	12 interface-split evaluations (4 interfaces $\times$ 3 splits); additional 0.171 ± 0.012 loss reduction
threshold-split probe	synthesized operator in a worked construction	visual type trace only; not used as a benchmark-result row

Table 20: Operator provenance in the reported evidence. The table separates fixed interfaces, evaluation-time reuse, a measured development-time language extension, and the pedagogical construction in Figure 1.

Proposal-Core Substitution

We vary the proposal endpoint while freezing each benchmark's task set, hypothesis language, executable interfaces, parser, and native evaluator. These are complete provider-substitution audits rather than parameter-scaling experiments: endpoint size and training data are not consistently disclosed, and no operator is added during evaluation.

DiscoveryBench substitution. Table 21 uses the same 239 test tasks, four proposals, one search round, frozen language, and GPT-4o HMS judge for all three endpoints. HMS spans only 0.56 points; Cons-HMS spans 1.14 points. Thus the full DiscoveryBench result is not tied to one proposal provider.

Proposal core	Tasks	HMS	Cons-HMS
GPT-4o-mini	239	28.70	34.56
Gemini 2.5 Flash Lite	239	**29.14**	35.00
Qwen3-30B-A3B-Instruct	239	28.58	**35.69**

Table 21: Complete DiscoveryBench proposal-core substitution under the matched frozen protocol. Each row contains all 239 native evaluator decisions.

NewtonBench substitution. We substitute five endpoints while holding the complete 324-task set, executable probe library, operator charts, parser, abstention rule, and symbolic-law judge fixed. Table 22 reports one complete run per endpoint. Correct-law counts vary by at most one task, despite substantial differences among the proposal models. In this controlled setting, most accepted laws are therefore determined by executable measurement and validation rather than by endpoint identity.

Proposal core	Answered	Coverage	Correct	SA-all	SA-answered
GPT-4o-mini	248	76.5%	94	29.01	37.90
Gemini 2.5 Flash Lite	248	76.5%	95	29.32	38.31
Qwen3-30B-A3B-Instruct	247	76.2%	95	29.32	38.46
GPT-4o	246	75.9%	95	29.32	38.62
DeepSeek V4 Flash	248	76.5%	94	29.01	37.90

Table 22: Full NewtonBench proposal-core substitution under one frozen diagnostic kernel. Coverage is the fraction of tasks on which a law is submitted. This diagnostic tests provider robustness; it is not a model-size-scaling experiment.

The absolute values in Table 22 are not substituted for the 49.38% headline result. The retained headline artifact is the frozen July 17 configuration with development-time promotion disabled; the proposal-core audit uses the later diagnostic kernel with its fixed charts and gate configuration. Mixing the two would confound a backbone change with an implementation change. The intended conclusion is narrower and directly supported: within the diagnostic kernel, the complete-task result is stable across all five proposal endpoints. Because provider parameter counts and training mixtures are not consistently disclosed, the table also makes no claim about scaling with model size.

UltraHorizon substitution. We also completed a second proposal-core substitution under the controller-enabled UltraHorizon protocol. All rows use the same 96 hard episodes (32 seeds in each environment), 50-step horizon, action budget of 220, disabled environment hints, enabled public terminal controller, and exact paper-style judge. Table 23 is deliberately separate from the 51.04 strict headline row, which disables that controller. It tests whether the same long-horizon state and execution protocol remains operational when the proposal endpoint changes.

Proposal core	Grid	Seq	Bio	Overall
GPT-4o-mini	54.38	81.25	90.47	**75.36**
Gemini 2.5 Flash Lite	80.63	100.00	15.63	65.42
Qwen3-30B-A3B-Instruct	33.13	37.50	3.28	24.64

Table 23: Complete UltraHorizon proposal-core substitution under the controller-enabled protocol. Each overall score averages Grid, Seq, and Bio over the same 32 seeds per environment.

The substitution exposes genuine environment–core interaction rather than a uniform scaling effect: the compact Gemini endpoint is strongest on Grid and Seq, whereas GPT-4o-mini is substantially stronger on Bio. More importantly for the architecture claim, all three endpoints use the identical typed state, executor, controller, and judge configuration; no environment-specific prompt or operator library is selected by model identity.

Artifact Manifest and Compute Accounting

The machine-readable map from every reported row to its complete-run JSON/JSONL/CSV artifact is `paper_assets/evidence/evidence_index.csv`. It indexes the 239-task DiscoveryBench run, the 324-task NewtonBench run, the 96-episode UltraHorizon run, DiscoveryBench controls, multi-step trajectory, bootstrap intervals, all five NewtonBench substitutions, the matched three-core DiscoveryBench audit, and the controller-enabled UltraHorizon substitutions. The archive retains per-task outputs and evaluator decisions; no table is reconstructed from manually selected examples. A separate machine-readable substitution audit validates all 11 rows against their task cardinalities, frozen protocol fields, and SHA-256 source hashes.

All headline rows use GPT-4o-mini for proposal calls. DiscoveryBench uses `openai/gpt-4o` as the native HMS judge, NewtonBench uses the repository `gpt41` symbolic-law judge, and UltraHorizon uses `DeepSeek-R1-0528-BF16` through the reproduced paper-style judge path. Legacy headline artifacts retain row counts, outputs, scores, and wall time but not call-level token records, so comparisons are task-, backbone-, and evaluator-matched rather than token-matched.

Local orchestration used Python 3.12.7 on macOS 15.2 with an Apple M1 Pro CPU and 16 GB unified memory; no local GPU training was performed. The retained environment uses NumPy 1.26.4, pandas 2.2.2, SciPy 1.13.1, and OpenAI Python client 1.109.1. Foundation-model inference was served through the named API endpoints, whose provider-side hardware is not exposed. Model identifiers, judge identifiers, seeds, task cardinalities, and complete outputs are stored with the evidence index.

Complete-run integrity checks. Every aggregate is keyed by the benchmark's native evaluation unit, and the packaging audit fails if a key is missing or duplicated. Table 24 records the cardinality and frozen controls checked before the corresponding row is admitted to the paper.

Benchmark	Units	Native key	Frozen controls
DiscoveryBench	239	dataset, metadata id, query id	test split, four proposals, one round, language, HMS judge
NewtonBench	324	module, difficulty, law version, system	module grid, executable charts, parser, abstention rule, law judge
UltraHorizon	96	environment, seed	32 seeds per environment, horizon, action budget, hints, controller mode, paper judge

Table 24: Machine-checked integrity conditions for complete reported runs.

Provider interruptions are resumed by native task key. Completed evaluator rows are immutable; a prediction written immediately before an interrupted judge call is accepted only after the matching evaluator row is produced. Aggregation then checks exact cardinality, unique keys, frozen-language state, and protocol equality before hashing the result. This permits transport-level recovery without selecting attempts by score or changing any scientific artifact after evaluation.